

PLUG & CHUG

AI Agents, Explained

If your AI assistant can already do the work,
what is a local workspace for?



@kwis7

Reader edition 4 / 2026-10-03

github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide

In this guide

Start with a task you know well. These chapters explain the decisions involved in setting up a workspace, with a fictional venue comparison you can use for practice.

Build an AI workspace you can return to	3
Before the first task: how an agent works	5
1. Finish one small task	12
2. Give the work a place to continue	14
3. Pick up where you left off	16
4. Keep a method that worked	18
5. Separate work when responsibilities collide	20
6. Use the workspace with another tool	22
7. Know what has actually been completed	24
8. Maintain what you use	26
9. Reference: the standard toolkit	28
Further reading	31

Build an AI workspace you can return to

Suppose you are preparing a report over several weeks. An AI assistant helps you compare sources and revise a draft. By the second week, you have more than a collection of answers: you have selected evidence, rejected a few claims, settled on a format and left questions for later. To continue well, you need to recover those decisions along with the draft. Otherwise, you may repeat work or quietly lose the reasons behind it.

This guide grew out of Computational Social Science research, where the relationship between sources, methods and conclusions matters. The same problem appears in studying, writing, teaching and many other recurring tasks. A personal AI system gives that work a structure you can inspect and revise. You decide what to keep, what the assistant may use and which methods are worth repeating.

You can build it with the AI application you already use. The guided setup assumes no programming background. As you work through the book, the assistant can handle the technical steps its tools permit and explain the choices involved. The purpose is to make your work easier to continue without requiring you to organise your life around the software.

Start with your own assistant

The [beginner guide](#) contains a starting prompt with the repository link. Give it to your usual assistant. It will ask about your work, app and computer, then propose a small setup. Review the proposed folder and its contents before agreeing to file creation. The proposal should account for the tools actually available: a chat-only assistant can explain a step for you to perform, while an assistant with appropriate file access may perform it directly.

Choose a task small enough to finish and check. At the end, you should have the result, a record of unfinished work and a short card telling you which files to open next time. Try that card in a new conversation and check that the assistant reads the named files, identifies the unfinished work and resumes at the agreed next step.

The chapters explain why each part is useful. When a term is unfamiliar, ask the assistant to explain it through your task. You can also use terminal commands directly; that optional route is collected in the final reference chapter and the [quick start](#).

Keep the work, change the model

The report example also explains why ownership matters. You should be able to inspect the sources and decisions, correct a procedure or try another model without surrendering the work you have already done. Keeping records in accessible formats makes those choices possible. Moving to another interface or provider still requires adaptation and checks; saving the files does not establish compatibility by itself.

The book uses **harness** for the configured environment around model execution: instructions, files, tools, task state and checks. Some of these are text that guides the model. Others are software that

controls access or checks an action. Their effects differ, so we will be careful about what each one can establish.

The company diagram in the next chapter separates these responsibilities. You are the owner who sets the purpose and authorises consequential actions. The model does the reasoning within the working arrangements you have chosen. This analogy describes roles; it does not imply human awareness, loyalty or automatic access to the company's records.

Learn through one example, then apply it to your work

Chapters 1-4 use three fictional venue notes to compare options, check a conclusion and leave the next step for another session. Because the inputs are supplied and contain no personal information, you can concentrate on the method. The exercise is optional. If you have already chosen a task during setup, apply the explanations to that task; there is no need to build a dedicated venue-comparison Agent.

Chapters 5-8 consider the decisions that follow: when to separate responsibilities, how to change tools, what to verify and how to maintain the system. The reference chapter then points to the standard toolkit. Read those sections when the corresponding question arises in your work.

The exercise needs only a few files. The guided route can create those teaching files directly; the standard generator creates a fuller scaffold with task manifests and local checks. Ask your assistant which arrangement suits the task and how it will be created.

Read evidence at its stated level

Throughout the example, we will distinguish what a source says, what the assistant concludes and what has actually been tested. The data is synthetic. Commands and compatibility records describe the repository's implementation, with their stated limits; they do not certify the app on your computer. Reading an instruction, passing a static check, resuming in a fresh session and delivering something externally each require their own evidence.

Continue with [how an agent works](#) for the company diagram and the concepts behind it. As you read, consider which records your own task would need for someone to resume it accurately.

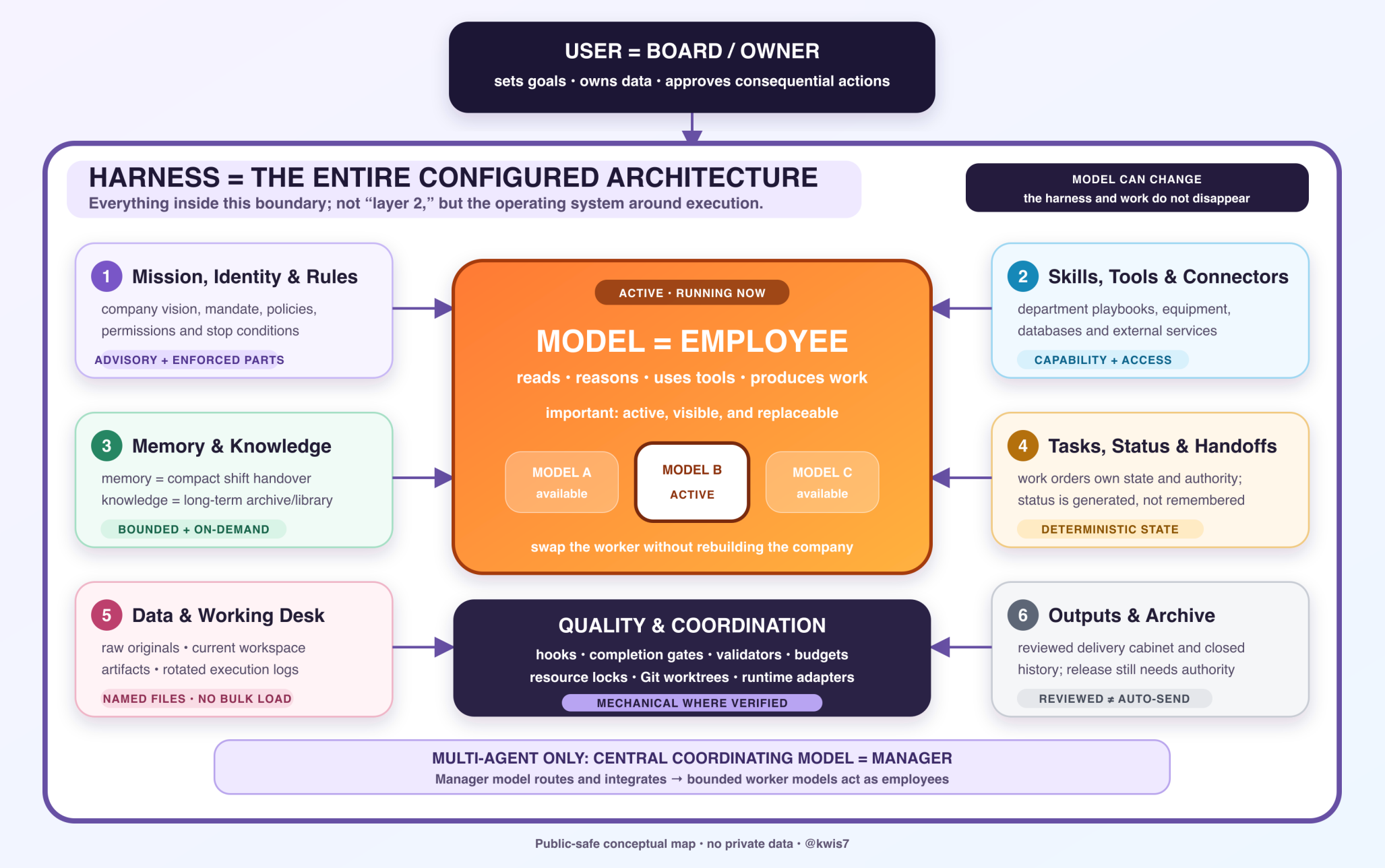
Before the first task: how an agent works

In the venue exercise, an assistant must read three notes, compare the requirements, write a result and leave enough information to continue later. The model interprets the notes and proposes what to do. Other parts of the system make the notes available, perform file operations and retain the result. Understanding these contributions helps you decide which part needs attention when the work goes wrong.

This book uses **agent** for a role carried out through a model, instructions, working state, available tools and an execution loop. Implementations vary. An agent may work entirely within one session, without local files or other agents. We introduce files because they help preserve the work in this example, not because every agent must have a folder structure.

Read the diagram as a working company

The company diagram separates responsibilities that can otherwise become confused. It shows the repository's complete design, including arrangements you may need only as the work grows. Begin by understanding the roles; you do not need to implement every box.



The active model inside the user's configured harness.

At the top, **you are the owner**. You choose the goal, control the materials and authorise consequential actions. For the venue task, that means specifying the workshop requirements and deciding whether anyone may contact a business. The **model**, shown as the active employee, interprets the supplied material and proposes responses or tool calls. Records, permissions and the model itself have separate roles in the design.

An **Agent is a working role**, such as a research assistant assigned to compare sources. Instructions define its assignment and boundaries, state records its progress, and tools supply the operations it can use. The same model can serve different roles in separate sessions with different context and permissions. A role can also be carried out by different models over time. Naming a role "Research Agent" does not train a separate model for it.

The **harness** is the configured working environment: rules, files, procedures, tools, state and checks. In the diagram, it and the active model form one system. If you replace the model, you can retain the records and methods around it. A change of runtime may nevertheless require different adapters and fresh verification. The persistent work gives you something to transfer; it does not make every tool interchangeable.

Other parts of the analogy help explain what to retain. A Skill resembles a playbook, a tool provides equipment, memory gives a short handover, and knowledge supplies reference material. A coordinating model has a management role when several responsibilities need routing or integration. That role has its own limits; it is neither required for a single task nor a higher source of authority.

The analogy describes responsibilities, without implying consciousness, loyalty, permanent memory or automatic access. Its quality controls also depend on implementation. A rule that says "do not send" guides behavior when read; software that blocks an unauthorised send enforces a boundary. The latter claim requires evidence that the corresponding control is implemented and works.

Keep the core terms separate

Term	Its job in the system
Model	Interprets supplied context and generates responses or proposed actions
Agent	Carries a role through instructions, state, tools, and an execution loop
Prompt	Communicates a request, instruction, or other material to the model
Skill	Packages a reusable procedure and supporting resources
Tool	Performs an operation, such as reading a file or running a calculation
Runtime	Hosts execution, supplies context, dispatches tools, and handles results
Harness	Configures the working environment around model execution
File	Stores information that can persist beyond a session
Context	The material actually available to the model for the current response

One application may provide several of these functions, including execution, tools, storage and the interface. The terms help us describe what happens; they do not prescribe a separate product for every row.

The execution loop also differs from a fixed workflow. A fixed workflow follows a predefined sequence. In an agentic loop, the model can choose a subsequent step after receiving an observation, within the system's limits. A practical system may combine the two: a fixed review process can surround work in which the model chooses which file to read next. This distinction follows [Anthropic's explanation of workflows and agents](#), without certifying any particular product's implementation.

A prompt sets the task; a Skill carries a method

A task prompt might say, "Compare the supplied venues for 16 people after 18:00, with step-free entry required." It defines the immediate request. A durable rule such as "keep unsupported facts unknown" applies across a defined set of tasks. Both can influence the model when loaded, but one describes today's assignment and the other expresses a continuing instruction.

A **Skill** records a procedure you want to reuse. In this repository, its instructions are saved in `SKILL.md`, sometimes with examples, references or scripts alongside them. A source-comparison Skill could explain how to extract requirements, cite the relevant notes and handle missing fields. The next comparison can then use the same method without your having to write it again.

This is the sense in which a Skill **pins a useful prompt or method**: it gives the procedure a stable place where it can be discovered. The [Agent Skills specification](#) uses the name and description for initial discovery, with full instructions loaded when needed. Discovery and loading depend on the

host. The document is not permanently active in every turn, and its filename does not make it a higher-priority system message.

When loaded, Skill instructions join the current context along with relevant rules, conversation and tool observations assembled by the runtime. Supplying this material changes the model's input, not its learned weights. It also leaves permissions unchanged. A script included with a Skill needs an actual execution tool and the relevant access; a procedure that mentions email neither connects an account nor authorises a send.

The difference is visible in the example. "Pick a venue" gives the model little basis for a judgment. The requirements, three source notes and comparison procedure give it evidence and a method to apply. That makes the work easier to evaluate, although the resulting comparison still needs review.

Follow one execution loop

The first exercise requests a comparison of synthetic venue notes. It authorises no booking. The runtime supplies that task and relevant instructions; the model identifies the requirements and the evidence it needs. If the notes have not yet entered context, their filenames provide locations to read, not facts about the venues.

The model can request a file-read tool by naming an available operation and its arguments. The runtime applies its configured access policy and dispatches the allowed operation. The result may contain the file, a denial or an error. Saying "I read the file" is merely text; evidence of execution comes from the actual operation and its result.

Returned content enters the subsequent working context. The model can now compare the notes or request another permitted operation. This cycle of proposing an action, receiving an observation and using it in the next step is the execution loop. A wrong path, unavailable tool, denied permission or incomplete result can interrupt it. A successful operation can still be interpreted incorrectly.

Once the notes are available, Cedar Hall's recorded facts support the requirements, Willow Room's hours fail the evening condition, and Maple Studio's step-free entry remains unspecified. Further reasoning cannot produce the missing entrance statement. The assistant can draft an unsent clarification question as a later task; it has neither obtained the answer nor received permission to contact anyone.

The draft goes to `workspace/comparison.md`. After source review and any corrections, the checked result goes to `outputs/comparison.md`, and `workspace/handoff.md` records actual progress and remaining limits. A successful file write establishes that content was saved, not that it was correct. With a chat-only interface, the assistant can return text for you to save and must leave automatic writeback unclaimed.

Finally, the assistant reports what was achieved and what remains. When it lacks necessary information or authority, stopping with a clear handoff may be the correct outcome. Chapter 1 lets you observe this process in a task small enough to check yourself.

What Markdown files actually do

The `.md` extension commonly denotes **Markdown**, a plain-text format you can edit in an ordinary text editor. A heading can begin with `#`, a list can use hyphens, and links can point to other files. This makes instructions and records accessible to you as well as to the assistant. You can inspect or revise the underlying text without depending on a particular AI application.

Filenames have effects only through the environment that uses them. A host may discover `AGENTS.md` as an instruction entrypoint or `SKILL.md` as a Skill entrypoint in an expected location. These conventions do not make the files universally executable. The project also uses names such as `RULES.md` and `workspace/handoff.md`; its workflow must arrange for the relevant contents to be read.

The following excerpts show how the same plain-text format can hold a request, a continuing rule, a procedure or current state. They are illustrations, not extra required files for the exercise or a complete installable Skill:

```
task-brief.md
Compare the supplied venues for 16 people after 18:00.
Step-free entry is required.

RULES.md (illustrative project rule)
Keep missing source facts unknown; do not invent them.

skills/source-comparison/SKILL.md (illustrative procedure)
Check each requirement against each source and cite the evidence.

workspace/handoff.md (example state, only if actually true)
Draft saved at workspace/comparison.md; source review remains.
```

Consider the handoff in that example. Saving it preserves text in a **file**. Reading it into the next session supplies that text as **context**. Neither operation changes the **model weights**, the learned parameters used to generate responses. These are three different forms of state. A future session can resume because the relevant records are retrieved, not because the saved facts have permanently become part of the model.

Personalise the work explicitly

Personalisation starts with a choice about your work. You may want a particular language, level of detail, evidence standard or form of review. Retaining a choice can save repeated prompting, provided you can see where it applies and change it later.

Suppose you tell the assistant, "For future source comparisons, lead with a short recommendation, then explain the evidence and unknowns." You have explicitly requested a continuing preference for comparison work. Its scope matters: that format may suit a research note and be unsuitable for a poem or debugging log. Record the request and its scope rather than inferring a general personality profile from the correction.

The exercise produces other information with different uses. "Maple's access remains unknown" belongs to current task state. A repeatable method for extracting, comparing, citing and reviewing

can become a Skill. An explanation of how missing evidence differs from a negative answer can become a knowledge note. Keeping these uses separate makes it easier to retrieve what a task needs and retire what has become stale.

Personalisation therefore changes the information and procedures supplied to the model. It does not train the model automatically. Repeated use alone cannot establish a guessed preference, and ordinary corrections should not be used to infer sensitive traits. Retain appropriate, authorised material and periodically check that earlier choices still suit the work.

Why build this alongside existing AI software?

If you use Codex, Claude Code or WorkBuddy, begin with the capabilities actually available there. The guide does not assume those products lack memory, Skills or workspace features. Use your chosen application's runtime, tools, instruction loading and storage. Building a personal system does not require writing another orchestration engine or custom application.

Your contribution is deciding how those capabilities serve your work: which role the assistant has, which authorised sources it uses, which preferences and methods apply, and which records explain the current result. The built-in arrangements may already be sufficient. Where they need supplementing, add something that solves an identifiable problem, such as recovering the evidence behind a report revision.

In this guide, **local** describes a durable workspace under your control. You can inspect its files, back them up and, where appropriate, keep versions that show what changed. Source attribution preserves the reasons for a conclusion, and a handoff helps another session recover the next step. A compatible tool can reuse the authorised material after its loading and saving behavior has been checked.

The location of these files does not determine where the model runs. A hosted model or connector may receive their contents during use, so privacy still depends on data flows, permissions, service settings and what you provide. A .md extension or folder name offers no privacy guarantee. Changing tools likewise requires testing how the new environment loads instructions and saves results.

This effort is most useful for recurring work, tasks that span sessions or results that need an inspectable evidence trail. Normal chat may be sufficient for a one-off explanation. A persistent workspace also needs care: instructions grow stale, copies diverge and procedures change. Begin with a useful task, then keep the parts that make its next occurrence easier. The [first-task chapter](#) gives you a small example to try.

1. Finish one small task

You are planning a workshop for 16 people after 18:00, and everyone must be able to enter without steps. Before contacting a venue, you want to know which options meet those requirements and which need more information. A short comparison is enough for this decision. A booking would require additional facts and separate permission.

The [first-project exercise](#) supplies three synthetic venue notes. These are fictional examples, not real businesses or verified live information. Use them to practise working from sources without browsing, sending messages, booking or making payments. None of those actions is authorised by the exercise.

If you have chosen your own task in the [beginner guide](#), follow the same method with its agreed inputs and result. The venue example shows the reasoning in a form you can inspect; it is optional practice.

Know the result you want

A ranking alone would conceal a crucial difference: one venue fails a requirement, while another has an unanswered question. Ask for a comparison that shows the source facts before explaining the judgment. Applying the same requirements to each candidate makes the conclusion easier to inspect.

Venue	Capacity	Recorded hours	Step-free entry	Judgment
Cedar Hall	18	18:00-21:00	Yes	Meets the recorded requirements
Willow Room	24	09:00-17:00	Yes	Fails the evening-hours requirement
Maple Studio	20	18:00-22:00	Unspecified	Cannot confirm accessibility

Cedar Hall meets all three requirements on the supplied facts. Willow Room has enough space, but its recorded hours end before the workshop begins. Maple Studio has enough space and evening hours; the note gives no accessibility information, so its suitability remains uncertain. Missing evidence of step-free entry does not establish that an entrance has steps.

These notes also leave other questions open. They do not establish availability on a particular date, the workshop's duration, price or booking terms. The comparison can identify a suitable candidate for further investigation. It cannot supply the missing facts needed for a booking.

Choose an available working route

Use an AI tool you already have and check what it can access. Direct file work requires permission to read the exercise files and write inside the working copy you choose. If your interface accepts only pasted text or uploads, supply the three notes and save the returned Markdown yourself. You can still examine the comparison, but filesystem loading and writeback remain manual and untested. Normal model or API usage costs may apply; the exercise needs no paid data service.

To preserve the bundled references, copy the exercise into a local practice folder and identify that copy as the place the agent may change. From the exercise root, give it this assignment:

```
Read task-brief.md, sources/cedar.md, sources/willow.md,
and sources/maple.md. Do not read expected/ yet.
Compare the fictional venues for 16 people after 18:00,
with step-free entry required. Use only the supplied notes.
Write workspace/comparison.md with source links, judgments,
and unknowns. Identify missing inputs instead of guessing.
Do not browse, contact, or book. Agree on any overwrite first.
```

Start by producing your own draft, with the reference answers still closed. In a chat-only interface, save it yourself and record that local file loading was not tested. The files `expected/comparison.md` and `expected/handoff.md` show what a checked exercise might contain. Their presence in the repository says nothing about whether your attempt succeeded.

Review before accepting

Open `workspace/comparison.md` and follow its source links to the three files under `sources/`. Check the capacities and time windows against the notes. Then examine how the requirements were applied: all three venues have enough capacity; Cedar and Maple have evening hours; Cedar alone also has a recorded step-free entrance. Once you have reviewed that reasoning, compare your result with `expected/comparison.md`.

If the draft recommends Maple without qualification, ask what establishes its accessibility. The note cannot answer that question. Correct the entry to "unknown," revise the recommendation and record the repair. This is a useful review habit: follow an important conclusion back to the specific fact it needs, rather than judging the answer by how convincing the prose sounds.

After any corrections, save the checked version at `outputs/comparison.md` while preserving the draft. The handoff at `workspace/handoff.md` should say what was checked, what remains unknown, which inputs and writes are permitted, and what to do next. Here the next step is to draft an unsent question about Maple's entrance. Drafting it does not obtain an answer or authorise sending it.

You now have a result and the information needed to continue. Recovery remains untested until you actually try it in another session. Likewise, a real booking decision would still require new information and separate authority. The next chapter explains how the files preserve these distinctions.

2. Give the work a place to continue

Imagine reopening the venue comparison a week later. The recommendation says Cedar Hall is suitable, but you also need to know what "suitable" meant: enough space for 16 people, hours after 18:00 and recorded step-free entry. You need to see the notes behind that judgment and remember that availability and booking terms were never checked. The answer alone cannot preserve all of this.

A workspace keeps the result together with the evidence and state needed to interpret it. For this exercise, a small folder is sufficient. Separate the supplied notes from your draft, checked result and handoff. Leave the bundled `expected/` directory as a teaching reference. You can establish these roles without adding a database, a new Agent or the full standard scaffold.

Give each file a clear job

The notes under `sources/` record the supplied evidence. Work on the comparison at `workspace/comparison.md`, then place the reviewed version at `outputs/comparison.md`. Keeping both makes the review visible, but it also introduces a responsibility: the old draft must not become a competing source of current conclusions. When new evidence changes the decision, review the revision, update the result and record its status in `workspace/handoff.md`.

The handoff answers a different question from the comparison. It tells the next session where the work stands and which step is permitted next. In our example, no venue has been contacted and Maple's accessibility remains unknown. A reader should be able to recover those facts without searching through the entire conversation.

Larger projects need the same distinctions, even if their files have different names:

Kind of information	Question it answers	Example
Source material	What evidence was supplied?	Cedar Hall's recorded hours
Task state	What is happening now?	Comparison reviewed; no venue contacted
Memory	What short pointer helps us resume?	Location of the active task and key decision
Knowledge	What stable understanding is reusable?	How missing evidence affects a comparison
Skill	What procedure should we repeat?	Compare each candidate against every requirement

These records become useful to a model when the runtime supplies them as **context** for the current session. A saved handoff may be available on disk yet absent from the model's input. For that reason, instructions about resuming should identify what to read, and the resumed answer should identify

the files and facts it used. Asking the agent to read a source establishes an instruction; observing the read and checking its use establish more.

Preserve sources and uncertainty

Maple's missing accessibility information belongs in the result as an unknown. Editing the source note to make the comparison complete would destroy the distinction between supplied evidence and your interpretation. If a new statement arrives later, preserve its source and date, then revise the affected judgment. The record will show why the answer changed without needing every sentence of the intervening chat.

Apply the same care to real research. A summary helps you understand a source, but a reviewer still needs to know which source supports the conclusion. Keep the attribution and the limits of the claim. Retain and expose only material you are authorised to use. Personal originals remain under your control, outside the Agent's read scope; provide the minimum authorised fields, an explicitly agent-readable redacted derivative or a controlled mediator output when such material is needed.

Let structure follow need

The standard scaffold offers named places for a larger project: `raw_data/` for public, synthetic or explicitly authorised nonprivate source inputs; `workspace/` for drafts; `artifacts/` for generated intermediates; `logs/` for execution traces; `outputs/` for reviewed results; and `archive/` for closed material. The [filesystem contract](#) defines their full responsibilities.

These names make the intended use easier to understand. Access still depends on the tools and permissions: a file named "private" remains readable to a tool with access to the whole directory. Sensitive work needs actual access restrictions and a deliberate decision about what enters the model. Public examples should use synthetic or authorised material, never disguised personal records.

Before creating another folder, try finding the current answer, its sources and the next action. If that takes less than a minute, the present structure may be enough. If it takes longer, identify the ambiguity: perhaps two drafts look current, or the handoff names a file that has moved. Repair that problem first. Add structure when it makes a specific part of the work easier to continue.

3. Pick up where you left off

The reviewed comparison leaves a specific job unfinished: finding out whether Maple has a step-free entrance. Another session can prepare that question if it can find the result, understand the gap and see what you have authorised next. You can name the handoff file and ask it to continue from there.

Try this by closing the original conversation and starting a fresh one. The saved files should support the next step on their own. Pay particular attention to the status of the work: Cedar is a candidate supported by the supplied notes, Maple's entrance remains unknown, and no venue has been booked. A promise from the original model to remember these details gives you less evidence than watching a new session use the records correctly.

Leave a handoff with a next action

Write the handoff while the outcome is still clear to you. Include the requirements, the files containing the result and its sources, the checks you performed, and the unresolved issue. Then specify the next permitted action. A later reader needs enough information to check the decision and continue; you can leave out the discarded wording and other drafting details. For example:

```
Task: compare three fictional venues for 16 people after 18:00;
step-free entry is required.
Result: outputs/comparison.md. Draft: workspace/comparison.md.
Sources: sources/cedar.md,
sources/willow.md, sources/maple.md.
Checked: capacities, hours, and accessibility statements.
Decision: Cedar meets the recorded requirements; Willow fails
the hours requirement; Maple's accessibility remains unknown.
Limit: synthetic notes only. Availability, price, and exact event
duration are unverified. No contact or booking is authorised.
Next: verify the reviewed result exists, then draft one unsent
question about Maple's step-free entrance in workspace/maple-question.md.
Read only this handoff, task-brief.md, the reviewed result,
and the three named sources. Write only the question and this handoff.
Recovery: not tested until a fresh session is observed.
```

The file paths let the next session inspect the basis for the decision. The limits explain why drafting a question is appropriate while contacting or booking a venue is not authorised. Compare your note with [expected/handoff.md](#), then make sure your own version describes checks you actually performed. The reference is a teaching example; copying it does not establish that those checks happened in your session.

Run the restart exercise

Open a fresh session in the practice copy and ask it to begin with `workspace/handoff.md`. If your interface requires you to supply the files, provide the handoff and its named sources explicitly. Have the session inspect only the permitted files and explain the current result and unresolved issue before continuing. Its next job is to save one unsent question about Maple's step-free entrance at `workspace/maple-question.md` and update the handoff. The exercise allows no browsing, contact or booking.

Check the explanation against the reviewed comparison. Cedar should still be the supported candidate, Willow should still fail the hours requirement, and Maple's entrance should still be unknown. Reopen the question and the updated handoff to see whether the files were saved as intended. Record the tool, date and observed outcome before marking recovery as tested. Saving that draft leaves Maple's entrance status unresolved. Any missing input should be reported as a gap; remembered facts cannot replace evidence the task requires the session to read.

You can also rehearse a restart with incomplete inputs. In another fresh session, use the tutorial's missing-source prompt: permit only `task-brief.md`, `sources/cedar.md` and `sources/willow.md`, and declare Maple's note unavailable. Leave the original note in place, exclude the other exercise files and remembered facts, and save the limited comparison at `workspace/missing-source-check.md`. Maple should be unassessed in this result, while the original reviewed comparison remains untouched.

That exercise shows whether the session follows the declared input scope. To establish that it cannot access the excluded files, you would also need evidence from the tool's permissions. Keeping a source out of the prompt does not itself make the file inaccessible.

Recover before repeating

After an interruption, inspect the draft, reviewed result and `workspace/handoff.md` before starting over. Suppose the draft exists but the reviewed result is missing. The next step is review, even if the draft looks finished. Finding the last successful step preserves useful work and prevents a partial result from being mistaken for a checked one.

Repeating a read or recreating a disposable draft is usually straightforward. External actions need more care. If a booking request times out, the response may have failed to reach you after the service accepted the booking. Check the external record before repeating the action; otherwise a retry could produce a duplicate booking, message or payment. Any new booking, message or payment also needs the appropriate authorisation. Our fictional exercise performs none of them, but its handoff gives you a place to learn how to record action status accurately.

For larger projects, the toolkit stores state in `tasks/<task-id>/task.yaml` and generates `STATUS.md` from it. The additional fields make scope, verification and lifecycle explicit. A small handoff can serve the first exercise: keep the current state in one place, preserve usable partial results, and identify the files and permitted action needed to continue.

4. Keep a method that worked

The venue comparison contains a procedure you could use again. For each candidate, extract the same fields, compare them with the requirements, and distinguish a failed requirement from missing information. A later comparison might concern suppliers or reading notes, but it could still benefit from those steps.

Once you have used a procedure enough to recognise what helps, save it where you can find it. A **Skill starts with a Markdown instruction document**, conventionally named `SKILL.md`. When the host loads that document, its text enters the model's context. The model uses its existing capabilities to follow the instructions; its trained weights remain unchanged.

Keeping a method in a discoverable place is sometimes described as pinning it for reuse. Discovery and loading still depend on the host. The name and description help with selection; the host can then load the full instructions and references as needed. Pinning does not mean that the full document is present in every turn or has higher instruction priority. Examples, templates and scripts may accompany the instructions, with scripts requiring actual tools and permission to run. Maintaining this package makes sense when repeated work or a recurring error gives you a reason to reuse it.

Extract the method, leave the case behind

To extract the method, ask what would remain useful if all three venue notes were replaced. Cedar's place on the shortlist depends on this exercise's requirements and evidence. Checking each candidate against every required condition is useful in the next comparison too. A procedure that includes the old recommendation could steer a new task towards Cedar before the new evidence has been considered.

Here is a sketch of the procedure worth retaining:

```
Name: Source-backed comparison
Use when: comparing supplied candidates against explicit needs.
Inputs: requirements and named source notes.
Method: extract facts with sources; check each requirement;
        separate meets, fails, and unknown; explain the final judgment.
Output: concise comparison, unresolved questions, and handoff.
Limits: do not infer missing facts or perform external actions.
```

Use the sketch to decide what the method should do. The repository's [skill template](#) shows how to package it. Check the selected runtime's requirements for installation and discovery; the sketch alone is not a format every tool can install.

Write a useful trigger

The description is what helps a host choose a skill before reading its full procedure. If it says only “Helps with research” , it could apply to summarisation, writing or data collection. Give it a more specific job by naming the work, the expected inputs and outputs, and the cases it excludes.

Test that description with requests you might actually make. “Compare these three supplier notes against our requirements” should select the comparison method. “Write a thank-you note” should not. A request to “Compare these venues and book the best one” needs to be separated into comparison and booking: the skill provides a method for the first, while the second requires its own capability and authority. Selecting the skill cannot supply that authority.

Next, inspect the output produced with the method. The venue fixture should support Cedar, exclude Willow because of its hours, and leave Maple's accessibility unresolved. Try a variation with a missing source or contradictory fact as well. The variation is useful because a reusable procedure must respond to the evidence it receives. Reproducing one reference answer gives you little information about how it handles a different case.

Put each lesson in the right place

Some lessons affect the procedure; others belong with the task or its owner. An instruction to begin future source comparisons with a recommendation is a scoped preference for the relevant operating rules. Maple's unresolved entrance belongs in the current task state. Stable background can go in knowledge, while the repeated comparison steps belong in the skill. Reading files or accessing a service remains the work of a tool.

With those purposes separated, a change is easier to make in the right place. The skill's description helps with discovery, its main text explains the method, and longer references can be retrieved when needed. There is no need to load every skill and reference into each conversation. Select the material relevant to the current work so the procedure stays understandable and the context remains manageable.

Share procedure deliberately

A second agent or project can use the comparison procedure with its own authorised inputs. Share the method and a synthetic example, while keeping private candidate notes, internal decisions and the full chat history with their owner. This lets another project try the useful steps without importing the original project's private material.

Return to the skill after actual use. If it repeatedly omits a required field, you have a concrete problem to fix and a result to check after the edit. An unusual request may need only a task-specific instruction. The [distillation guide](#) explains how to retain repeated experience and review procedures that overlap.

5. Separate work when responsibilities collide

You may eventually have several kinds of recurring work: analysing sources, preparing a report and drafting messages about the result. One agent with suitable skills can handle much of this. Before adding another, look for a problem that separate responsibility would solve.

Suppose the analysis is still changing while a designer prepares the final report. You need an agreement about who settles the claims, which version goes to the designer and who approves a revision. Separate owners may help manage that handoff. An inconsistent table format is a smaller problem that a template or skill can usually address. A different subject or format alone gives you little reason to introduce another agent; distinct sources, data boundaries, authority or work cycles provide stronger reasons.

Choose the right unit

Need	Usually start with
One temporary objective	A task or project
A procedure repeated across tasks	A skill
Stable reusable reference material	A knowledge collection
Recurring work with distinct ownership	A domain agent
One bounded independent contribution	A temporary subagent
Access to a service or executable action	A tool or connector

A domain agent maintains a continuing area of work through instructions, working files and access arrangements. The model carrying out that role can change. Ownership of the sources and authority to act should remain governed by the working agreement, rather than changing with the model selection.

The venue exercise needs only one agent. A recurring event workflow might later assign source comparisons to a research owner and approved messages to a communications owner. Those roles need different authority: comparing the notes gives a basis for a shortlist, while contacting a venue requires permission to send a particular message. Giving the roles separate names makes the arrangement easier to explain, but access restrictions still depend on tool permissions and explicit action rules.

Define ownership before delegation

Give a delegated task enough detail to be checked when it returns: the result you need, the inputs the worker may read, where it may write, the actions it must avoid and the review criteria.

“Research this” leaves each of those decisions open to interpretation.

For the venue exercise, you could ask a temporary reviewer to inspect the three fictional notes and report unsupported claims in `workspace/comparison.md`. It needs access to those files and a place to save its review. Rewriting the source notes, changing the scope or contacting venues would be outside that job. The lead then integrates the findings, checks the final answer and saves it to `outputs/comparison.md`.

The review earns its additional time and context when it can catch a plausible mistake. A statement that Maple meets the entrance requirement might look reasonable in a fluent report; a reviewer checking the note should notice that the information is missing. A second agent that simply rereads the conclusion and shares its assumptions would provide much less assurance. Give the reviewer evidence and a question it can investigate independently.

Keep one writer per authority

Shared reading is often useful. Shared editing needs an arrangement to prevent one contributor from overwriting another. If the lead and reviewer both revise the current comparison, it can become unclear which changes were accepted and which version was checked. Assign one writer to each authoritative file, with other contributors saving separate proposals or artifacts for that writer to integrate.

For concurrent code work, Git worktrees put changes in separate checkouts. Resource locks coordinate use of a shared file, device, port or other resource. Changes in separate checkouts may still use the same account, and a lock provides no privacy boundary. Neither grants permission to publish or perform external actions. Delegating a role likewise needs actual access controls if the worker is meant to be unable to read particular material.

When several continuing owners depend on one another's work, a control center can route tasks, manage those dependencies and integrate results. It needs to know where the work belongs. That coordinating responsibility gives it no additional ownership of private domain facts. Methods can be shared between agents; any transfer of data must stay within its authorised scope.

Test the boundary in ordinary language

Try the proposed descriptions on ordinary requests. Write two that clearly belong to an agent, two that do not and one that overlaps with a neighbour. Check who receives each request and who would integrate an overlapping task. The [routing guide](#) provides a fuller method.

If the new role has no distinct responsibility you can explain through those requests, keep the work with its current owner for now. You can split it when a specific source, authority or handoff problem arises. However many contributors take part, keep one accountable lead for the final result.

6. Use the workspace with another tool

Your saved venue notes, comparison and handoff give you material to work with in another tool. You can read them yourself and supply the authorised parts to a new model. What needs checking is how the new environment receives that material, uses its tools and saves the next result. File portability provides a starting point for those checks.

A move may involve several changes. The **model** does the reasoning, while the **provider** supplies inference through a service or backend. The **runtime** hosts the session, assembles context, exposes tools and may manage permissions or hooks. If you keep the same runtime and change a model setting, some of that arrangement may remain in place. Moving to another runtime can require different instruction entrypoints, tool configuration and recovery steps.

A **switchboard** can route model or provider configuration. Your task records and working files still need their own owner and location. Identifying which part you are changing helps you decide what can be carried forward and what must be adapted.

Identify what actually needs to move

For the venue task, gather the requirements, authorised sources, current result, handoff and relevant procedure, along with the operating rules that apply. These are the materials the next session needs. A destination's ability to accept many files is no reason to upload a whole personal workspace. Personal originals remain under your control; provide only the minimum fields authorised for the task, an explicitly agent-readable redacted derivative or a controlled mediator output.

Check how the selected tool takes in those materials. A local tool may discover designated entrypoints. A hosted workspace may need project instructions and selected uploads. A custom API application would have to assemble context and implement saving itself. You can use the facilities of an existing app when they meet your needs. An instruction to read files still requires an interface with actual file access.

Agree on where revisions return. If a new comparison appears only in chat, the authoritative local file will still contain the old result until someone updates it. Manual writeback is a workable choice; tool-based saving is another when the environment supports it. In either case, identify the current working copy and who updates it. Otherwise both copies can continue to change independently. Also confirm the destination and permission before supplying material to another service: local storage alone does not establish offline processing.

Read support as evidence, not a promise

The repository's [compatibility manifest](#) records product categories, generated configuration, limitations and evidence status. The recorded review date is 2026-08-09. Codex, Claude Code and Gemini CLI are classified there as `verified_static`, with fresh-session verification marked `not_run`. Those labels describe the evidence retained in the repository. They do not certify current vendor behavior.

The remaining entries distinguish documented integrations, manual projections, provisional support, providers, switchboards and custom-harness patterns. Read the entry for the kind of tool you are using, then consult the [adapter selection guide](#) and the selected adapter. An adapter describes the intended setup; observed loading and enforcement require separate evidence. Recheck the product's official instructions when installing or changing the integration.

Test the destination with a small fixture

Keep meaningful work in its original location while you try the synthetic venue exercise in a clean destination session. Check four separate behaviors:

1. The session identifies the intended rules and named source files.
2. Its comparison follows the supplied notes and leaves Maple's accessibility unknown.
3. The result and handoff are saved where expected, or the workflow clearly assigns saving to you.
4. If the integration claims a mechanical gate, an intentionally invalid completion is blocked and a valid completion passes.

Record the tool version, date, fixture, observed result and remaining gaps. For the gate check, look for the integration's actual behavior. A model saying that it understands the rules cannot establish that the gate ran. If no gate is connected, you can still review a comparison manually, with that responsibility and limitation recorded.

Retain the original working copy until the new route has passed the checks it needs. If the destination cannot save a file or recover the task, you can return to the working setup and address the specific gap. The evidence and procedures already saved remain available while you adapt the parts that depend on the runtime.

7. Know what has actually been completed

The venue exercise ends with a reviewed comparison and a usable handoff. Each judgment should follow the supplied notes and the stated requirements, with unknowns still visible. Cedar can be shortlisted on that basis. Date-specific availability and booking remain unconfirmed, so the next session must still understand that no workshop location has been secured.

This illustrates why completion needs to be defined at the start. A saved draft, a reviewed answer and a successful external action are different results. If you know which result the task requires, you can choose the checks that would support it and report how far the work actually got.

Match the check to the claim

Claim	Evidence to look for
A result was saved	Open the exact file and inspect its contents
A comparison is supported	Trace material facts and judgments to the sources
A script passed	Record its command, exit status, and relevant output
A document is usable	Render it, inspect pages, and test navigation
A runtime loaded instructions	Observe a fresh session using the intended endpoint
A completion gate works	Observe invalid and valid cases in that runtime
An external action succeeded	Check the service's authoritative result or readback

A **receipt** keeps a record of a check you performed: the date, inputs, relevant tool version, result and location of the evidence. Listing a command in the task file records an intention. After running it, you can record the outcome. A short review note may be enough for the venue comparison; work with greater consequences may need logs, hashes, screenshots or a separate reviewer.

Consider what an automated file check would tell you here. It could establish that the comparison exists or that the status page is current. The document could still contain a fluent claim that Maple has step-free access. Detecting that mistake requires checking the judgment against the source. Searching for the word "accessibility" would find the mistaken sentence as readily as the correct one.

Separate instructions from enforcement

The instruction to stop before booking gives the model a rule to follow. A restricted tool policy can also make the booking action unavailable. A validator has a different job: it can reject a task record when required evidence is missing. You can use these mechanisms together, provided you understand which part of the workflow each controls.

The repository's closeout gate checks structural conditions: output presence, declared verification state, receipt paths, current status, budgets and released locks. A task can satisfy those conditions while its report still misinterprets a source. Content review is needed to detect a mistaken or dishonest passed record; the gate does not independently determine whether every claim is true.

To use the gate during task completion, the runtime must be connected to it. A runtime hook can act on the validator's result when the integration is loaded and behaves as intended. Running the script in a terminal and seeing an error establishes that the script rejected that case. It leaves open whether the host would call it and block a final completion claim.

Test that connection in a fresh session with an intentionally incomplete task, followed by a valid one. Save the observed host behavior separately from the static checks. If the connection has not been tested, keep that part of the completion claim unverified.

Make partial completion useful

If Maple's source cannot be read, state which sources were checked and leave Maple unassessed. If the note was read but omitted the entrance information, record accessibility as unknown. Both situations leave a gap, but they require different next inputs. The handoff should make that difference clear.

Whether the gap prevents completion depends on the task. A comparison limited to the supplied evidence can correctly leave accessibility unknown. A task requiring confirmed accessibility for every option cannot finish with the same gap. Use the original completion criteria to decide; changing them merely to obtain a pass would obscure what remains undone.

When work has to stop, preserve the useful files and describe the dependency and next action. An honest failed or cancelled outcome can retain checked material for later use. Inventing successful receipts would hide the reason the task stopped and give the next session an unreliable starting point.

In the handoff, distinguish results that can be reused from checks that still need to be performed. Explain what evidence would allow the work to continue. The [reliability reference](#) gives further detail on tasks, receipts, gates and recovery.

8. Maintain what you use

After several weeks of use, you may have two versions of a useful procedure, a status page that still lists a finished task, and a memory file full of details from old conversations. The files were saved for good reasons, yet they now make it harder to determine what is current. Maintenance resolves this accumulated ambiguity.

Begin with something you had difficulty doing. Perhaps you could not find the source behind a conclusion, or a new session followed an obsolete instruction. Identify which record should have helped and why it failed. Decisions, provenance, reusable methods and recovery pointers are worth retaining when their purpose is clear. Material that no longer serves its current purpose may need to move, be archived or be retired.

Close the task cleanly

The venue exercise leaves a reviewed answer, named sources, a verification note and a handoff. Together they explain the outcome and what remains unknown. Keep those records distinguishable from the drafting attempts. If reviewing Maple's accessibility led to a useful general lesson, consider adding it to an existing Skill or knowledge note, where another comparison can retrieve it.

The memory file need not change after every task. Its purpose is to provide a short recovery aid: a durable decision or a pointer that would otherwise be difficult to find. Putting every draft and correction there makes that aid harder to use. Current progress belongs with its task; a substantial explanation belongs in knowledge or a referenced artifact.

In the standard scaffold, the task record is also the source for generated `STATUS.md`. Update the record and regenerate the page so that both describe the same state. Independently rewriting status in several places creates competing accounts of progress. At closeout, state the final outcome and its limits, and make clear which result was reviewed. A later reader should be able to distinguish completed work from an assignment still in progress.

Review friction, not appearances

Review often enough to notice problems while you can still understand their cause. A frequently used system might benefit from a brief weekly review. A quiet project may need review when you reopen it. Writing an interval in a document does not start a scheduled job; an automation must be configured and tested before you describe it as enabled.

Use repeated friction as evidence for a change. If you keep explaining the same background, examine the recovery pointers. If the wrong Skill keeps being chosen, examine its description and its overlap with other procedures. Lost sources, oversized startup files, skipped verification and contradictory copies each suggest a different repair. A confusing description may need one clearer sentence rather than another Agent or framework.

For a substantial change, record the problem and the intended effect. For example, new sessions may have been reading the entire archive even when only the active task was relevant. You could

change the endpoint to identify that task and retrieve older notes as needed. Then try a fresh session and observe what it reads. The edited text establishes the instruction change; the new session provides evidence about its effect.

Keep startup context bounded

The toolkit checks budgets for instruction files, memory, task records and large-file indexing. These limits encourage selecting what is useful at startup. A file below its limit can still contain irrelevant material, and filling the available space is not a performance goal.

When a size check fails, first ask what each section does. A stable explanation may belong in knowledge; detailed progress belongs with the task; closed history may belong in an archive. Leave concise pointers where the next session begins. Shortening must preserve the information needed to understand the work, especially the only record of an unresolved decision. Passing a size check by losing that record would make recovery worse.

For large inputs, a manifest can record paths, sizes, purposes and loading policies. This lets the model inspect an index and request the relevant portion instead of loading all the material at once. The manifest describes the intended use; actual access controls must still protect restricted material. Listing a path never grants permission to read it.

Change structure reversibly

A structural change can affect work beyond the file you are editing. Before merging Agents, retiring a Skill or moving an authoritative record, identify its owner, the references that point to it, active tasks that depend on it and the way to recover if the change fails. Preserve a recoverable version where appropriate. Obtain the required authority for destructive or external actions; a maintenance review does not grant it.

The [review and renewal guide](#) develops this review for systems with enough real use to evaluate. Judge a change by whether the next task can find the right records, follow the right method and continue accurately. A smaller arrangement may serve those needs better than a tidy but needlessly elaborate one.

9. Reference: the standard toolkit

The earlier exercise needs only a few files to compare sources and resume the work. A larger project may need task contracts, generated status, budgets or locks, especially when it has several responsibilities to coordinate. The Portable Agentic System toolkit creates a standard scaffold with those features, domain folders and adapter configuration. Use that setup route when its features serve the work. Completing the exercise does not require it, and the small teaching workspace is not a special generator mode.

This chapter explains the commands and what their results establish. Run the examples from this repository's root with an installed `python3` compatible with the source. First inspect the scripts and their `--help`, your environment and the proposed target. For an initial trial, choose a new disposable directory so that the generated files stay separate from existing work. The examples describe operations you can perform; they are not a record of execution on your computer.

Preview and create a scaffold

Begin with the [starter configuration](#). Its two owners illustrate how to describe responsibilities; they are not a recommendation to organise every project that way. For each Agent, the purpose, routing description, exclusions and examples should make it clear when work belongs there and when it belongs elsewhere. Copy the configuration to a working file before changing its sample domains.

Choose an unused target path. The first command previews where the scaffold and Agent folders would go:

```
python3 skills/portable-agentic-system/scripts/create_agentic_system.py \
--root /tmp/pas-learning-demo \
--config skills/portable-agentic-system/pas/examples/starter-config.json \
--dry-run
```

With `--dry-run`, the generator returns JSON containing the target root and Agent paths. It does not list every file it would write. Review the reported locations together with the templates and the generator's scaffold behavior. Once you have reviewed and authorised creation, run the same command without `--dry-run`.

By default, an existing target file causes the generator to refuse that write. Inspect a collision before deciding what to do; earlier writes may already have occurred. The `--force` option bypasses the refusal and permits overwriting, so it requires a deliberate decision about the existing work. It is not a routine way to make an error disappear.

The generated files include runtime configuration and operational scripts under `.pas/bin/`. Examine the hooks before trusting or enabling them in your chosen tool. At this stage you have created configuration files. Establishing that the runtime discovers and executes them requires a separate test.

Run static checks

Once the files exist, run the static checks below against the generated root. If you selected another target, substitute it consistently in each command.

```
python3 skills/portable-agentic-system/scripts/validate_agentic_system.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_budgets.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_descriptions.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/generate_status.py \
/tmp/pas-learning-demo --check
python3 skills/portable-agentic-system/scripts/harness_health_check.py \
/tmp/pas-learning-demo --json
```

A failure identifies something to investigate before continuing. For example, changing a task record can leave `STATUS.md` out of date. The `--check` invocation compares the page with the task records without rewriting it. Run `generate_status.py` against the same root without `--check` to regenerate the page, then check again. This keeps the task record as the source of current state.

You can also inspect the selected adapter's configuration:

```
python3 skills/portable-agentic-system/scripts/adapter_smoke.py \
/tmp/pas-learning-demo --runtime codex
```

The filename `adapter_smoke.py` can suggest a live test, but this probe inspects generated configuration. It reports `verification_level` as `static`. A passing result supports that inspection, without showing that an installed runtime loaded the instructions or enforced a hook. Choose the runtime you intend to use and follow its adapter's fresh-session procedure to obtain evidence for those behaviors.

Understand task authority

A standard `tasks/<task-id>/task.yaml` makes the task's agreement explicit. It identifies the task and owner, objective and scope, permitted inputs and writes, available tools and prohibited actions. It also records outputs, completion conditions, verification and receipts, execution resources and the handoff. These fields let the local checks examine the recorded state rather than infer completion from a conversational claim.

The generator saves JSON syntax inside this YAML-named file. Its standard-library parser reads that syntax and also accepts a conservative YAML subset. The `.yaml` extension therefore does not mean every YAML feature is supported. Use the generated task as a format example, and consult the [governance reference](#) before writing one manually. The short teaching handoff in chapter 3 serves recovery; it lacks the complete schema of a valid standard task manifest.

For an existing task in the generated system, run its bundled closeout gate from that system's root:

```
python3 .pas/bin/closeout_gate.py . tasks/T-123/task.yaml
```

Replace T-123 with the actual task ID. To accept a successful terminal closeout, the gate requires declared outputs, the required verification state and receipt paths, current generated status, compliance with budgets, released task locks and a handoff. A failed or cancelled task should retain its honest unsuccessful outcome. Closing its record does not turn it into a success.

These checks can identify a missing output or receipt, but the existence of a file does not establish its quality. Review the deliverable and the evidence substantively. Likewise, calling the gate directly does not show that the host invokes it or blocks an attempted completion. That behavior needs its own runtime evidence.

Find deeper implementation detail

Topic	Maintained reference
File authority and loading	Filesystem contract
Private material and external actions	Privacy boundaries
Routing and descriptions	Routing evaluations
Runtime classification	Compatibility manifest
Adapter selection and testing	Adapters
Tasks, gates, and concurrent work	Reliability reference
Full design interview	Build workbook

For repository development, run the Python suite with `python3 -m unittest discover -s tests -v`. Report a passing result as evidence for the local behavior covered by those tests. Fresh-session compatibility, publication, delivery and external transactions are separate outcomes, each requiring evidence from the relevant operation.

Further reading

These links open the source material and technical references cited in the book. Compatibility records describe the checks performed on their stated dates; check the current product documentation before setting up an integration.

beginner guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/docs/start-here.md>

quick start

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/QUICKSTART.md>

Anthropic's explanation of workflows and agents

<https://www.anthropic.com/engineering/building-effective-agents>

Agent Skills specification

<https://agentskills.io/specification>

first-project exercise

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/README.md>

filesystem contract

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/filesystem-contract.md>

expected/handoff.md

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/expected/handoff.md>

skill template

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/templates/skill/SKILL.md>

distillation guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/skill-distillation-and-fusion.md>

routing guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/description-routing-evals.md>

compatibility manifest

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/compatibility/runtime-compatibility.json>

adapter selection guide

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/references/adapters.md>

reliability reference

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/references/textbook-reliability.md>

review and renewal guide

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/references/system-review-and-renewal.md>

starter configuration

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/examples/starter-config.json>

governance reference

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/docs/reference/task-lifecycle.md>

Privacy boundaries

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/references/privacy-boundaries.md>

Build workbook

<https://github.com/kwis7/Plug-And-Chug-Agentive-Building-Guide/blob/main/skills/portable-agentive-system/pas/references/textbook-build-workbook.md>